DSAA2043 Exercise 6: Graph Algorithms

Problem 1. Consider the directed graph $G = (V, E)$ with vertices $V = \{1, 2, 3, 4\}$ and edges $E = \{(1, 2), (1, 3), (2, 3), (3, 4), (4, 1)\}$.

(a) Provide the adjacency list and adjacency matrix representations of this graph.

(b) Perform a Breadth-First Search (BFS) on G, starting from vertex 1. List the vertices in the order they are visited. Assume neighbors are visited in increasing order of their vertex number.

(c) Perform a Depth-First Search (DFS) on G, starting from vertex 1. List the vertices in the order they are first visited. Assume neighbors are visited in increasing order of their vertex number.

(d) Is this graph strongly connected? Briefly explain why or why not.

Solution:

(a) Graph Representations

(i) Adjacency List:

```
1 -> [2, 3]
2 -> [3]
3 -> [4]
4 -> [1]
```

(ii) Adjacency Matrix:

	1	**2**	**3**	**4**
1	0	1	1	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0

(b) Breadth-First Search (BFS)

1. Start at vertex **1**. Add it to the queue. Visited order: [1]. Queue: [1].
2. Dequeue 1. Visit its unvisited neighbors in order: 2, then 3. Visited order: [1, 2, 3]. Queue: [2, 3].
3. Dequeue 2. Its only neighbor is 3, which has already been visited. Queue: [3].
4. Dequeue 3. Visit its unvisited neighbor: 4. Visited order: [1, 2, 3, 4]. Queue: [4].
5. Dequeue 4. Its only neighbor is 1, which has already been visited. Queue is now empty.

The final visited order is: **1, 2, 3, 4**.

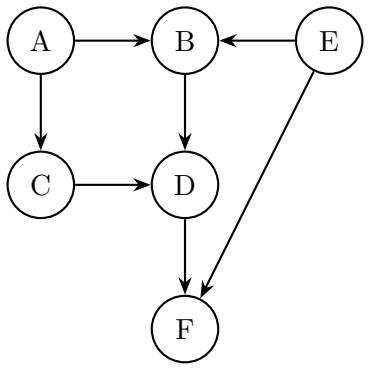
(c) Depth-First Search (DFS)

1. Start at vertex **1**. Mark visited. Visited order: [1].
2. Explore neighbor 2. Move to **2**. Mark visited. Visited order: [1, 2].
3. Explore neighbor 3. Move to **3**. Mark visited. Visited order: [1, 2, 3].
4. Explore neighbor 4. Move to **4**. Mark visited. Visited order: [1, 2, 3, 4].
5. Explore neighbor 1. It is already visited. Backtrack.

The final visited order is: **1, 2, 3, 4**.

(d) Connectivity Yes, the graph is strongly connected. A directed graph is strongly connected if for every pair of vertices (u, v), there is a path from u to v and a path from v to u. In this graph, the vertices $\{1, 2, 3, 4\}$ form a cycle $1 \to 2 \to 3 \to 4 \to 1$. This cycle ensures that every vertex can reach every other vertex.

Problem 2. Consider the following Directed Acyclic Graph (DAG). Provide one valid topological sort of the vertices.



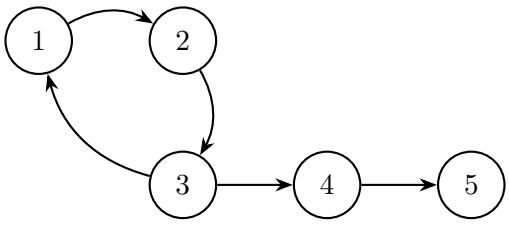
Solution:

We use Kahn's algorithm (based on in-degrees).

1. **Initial In-degrees:** A:0, B:2, C:1, D:2, E:0, F:2.
2. **Initialization:** Queue starts with nodes of in-degree 0: Queue = [A, E].
3. Dequeue A. Sorted: [A]. Decrement neighbors (B, C). In-degree(C) becomes 0. Enqueue C. Queue: [E, C].
4. Dequeue E. Sorted: [A, E]. Decrement neighbors (B, F). In-degree(B) becomes 0. Enqueue B. Queue: [C, B].
5. Dequeue C. Sorted: [A, E, C]. Decrement neighbor (D).
6. Dequeue B. Sorted: [A, E, C, B]. Decrement neighbor (D). In-degree(D) becomes 0. Enqueue D. Queue: [D].
7. Dequeue D. Sorted: [A, E, C, B, D]. Decrement neighbor (F). In-degree(F) becomes 0. Enqueue F. Queue: [F].
8. Dequeue F. Sorted: [A, E, C, B, D, F]. Queue is empty.

One valid topological sort is: **A, E, C, B, D, F**.

Problem 3. Execute Tarjan's algorithm on the following directed graph to find its Strongly Connected Components (SCCs). Start the DFS from vertex 1. Assume neighbors are visited in increasing order.



Solution:

The final SCCs are: **{1, 2, 3}**, **{4}**, **{5}**. The execution trace shows that nodes 5 and 4 are identified as roots of their own SCCs. Node 1 is identified as the root of the SCC {1, 2, 3} because its lowlink value equals its index after all its descendants have been visited, and the back-edges from 3 to 1 and within the component have updated the lowlink values of nodes 2 and 3 to point back to 1.

Problem 4. This question explores the principles behind Tarjan's algorithm for finding Strongly Connected Components (SCCs).

(a) Explain the significance of the `lowlink` value of a vertex v. Why does the condition `index[v] == lowlink[v]` indicate that v is the "root" of a new SCC?

(b) Consider the directed graph from Problem 3. Add exactly **one** directed edge to merge all vertices into a single SCC.

Solution:

(a) Significance: The `lowlink[v]` value is the smallest index of any node reachable from v (including itself) through the DFS tree, possibly followed by one back-edge. If `index[v] == lowlink[v]`, it means v cannot reach any node discovered before itself. This makes v the "entry point" or root of an SCC.

(b) Edge to Add: (5, 1). Reason: The current SCCs form a chain: $\{1, 2, 3\} \to \{4\} \to \{5\}$. Adding $(5, 1)$ creates a cycle connecting the last component back to the first, making all nodes mutually reachable.

Problem 5. Let $G = (V, E)$ be a directed simple graph stored in the adjacency-list format. Define $G^{rev} = (V, E^{rev})$ be the reverse graph of G, where $E^{rev} = \{(v, u) | (u, v) \in E\}$. Design an algorithm to produce the adjacency list of G^{rev} in $O(|V| + |E|)$ time. You can assume that $V = \{1, 2, \ldots, n\}$.

Solution:

The goal is to create a new adjacency list where for every edge $u \to v$ in the original graph G, we add an edge $v \to u$ in the reverse graph G^{rev}.

Algorithm:

1. **Initialization:** Create a new, empty adjacency list for the reverse graph, let's call it `adj_rev`. This will be an array of $|V|$ empty lists. This takes $O(|V|)$ time.
2. **Iterate and Reverse:** Iterate through each vertex u from 1 to $|V|$.
3. For each vertex u, iterate through all of its neighbors v in its original adjacency list `adj[u]`.
4. For each such neighbor v (which corresponds to an edge $u \to v$ in G), add u to the adjacency list of v in our new structure. That is, perform `adj_rev[v].add(u)`.

After iterating through all vertices and all their edges, `adj_rev` will be the complete adjacency list for G^{rev}.

Complexity Analysis:

- Step 1 (Initialization) takes $O(|V|)$ time.
- Steps 2-4 involve a nested loop. The outer loop runs $|V|$ times. The inner loop runs for a number of times equal to the out-degree of each vertex u. The sum of all out-degrees in a directed graph is equal to the total number of edges, $|E|$.
- Therefore, the total time spent in the nested loops is proportional to the sum of the lengths of all adjacency lists, which is $O(|E|)$.
- The total time complexity of the algorithm is the sum of these parts: $\boldsymbol{O(|V|) + O(|E|) = O(|V| + |E|)}$.

Problem 6. Let $G = (V, E)$ be a DAG, where each vertex $u \in V$ carries an integer weight denoted as w_u. Let $R(u)$ be the set of vertices in G that u can reach (i.e., for each vertex $v \in R(u)$, G has a path from u to v; note that $u \in R(u)$). Define $W(u) = \min_{v \in R(u)} w_v$. Design

an algorithm to compute the $W(u)$ values of all $u \in V$ in $O(|V| + |E|)$ time. (Hint: dynamic programming).

Solution:

This problem asks for the minimum weight among all vertices reachable from u, for every u. This can be solved efficiently using dynamic programming on the DAG.

Dynamic Programming Recurrence: The value $W(u)$ for a vertex u depends on its own weight w_u and the minimum weights reachable from its direct successors. Let $\text{Out}(u)$ be the set of out-neighbors of u. The recurrence relation is:

$$W(u) = \begin{cases} w_u & \text{if } \text{Out}(u) = \emptyset \text{ (u is a sink)} \\ \min(w_u, \min_{v \in \text{Out}(u)}\{W(v)\}) & \text{otherwise} \end{cases}$$

This formula works because any vertex reachable from u is either u itself, or is reachable from one of u's direct neighbors v.

Algorithm: The recurrence shows that to compute $W(u)$, we first need the W values for all of u's successors. This suggests we should process the vertices in a reverse topological order.

1. **Topological Sort:** Compute a topological sort of the vertices in G. This can be done using Kahn's algorithm or depth-first search in $O(|V| + |E|)$ time. Let this ordering be T.
2. **Reverse Order:** Iterate through the vertices of the graph in the **reverse** of the topological order T.
3. **Compute W(u):** For each vertex u in this reverse order:
   - Initialize a variable `min_successor_W` to infinity.
   - For each neighbor v in u's adjacency list (`Out(u)`):
     - Since we are processing in reverse topological order, $W(v)$ has already been computed.
     - Update `min_successor_W = min(min_successor_W, W(v))`.
   - Compute $W(u) = \min(w_u, \text{min\_successor\_W})$.

Complexity Analysis:

- Step 1 (Topological Sort) takes $O(|V| + |E|)$ time.
- Step 2 and 3 involve iterating through all vertices once. For each vertex u, we iterate through its outgoing edges. Over the entire loop, each vertex and each edge is processed exactly once. This step also takes $O(|V| + |E|)$ time.
- The total time complexity is $\boldsymbol{O(|V| + |E|) + O(|V| + |E|) = O(|V| + |E|)}$.